

Docker and Ansible - Technical Notes

Docker Overview

Docker is a popular platform that enables developers to create, deploy, and run applications in containers. Here are some of the most commonly used Docker commands with examples:

`docker run` - This command is used to run a container.

Example: `docker run hello-world`

`docker ps` - This command is used to list all the running containers.

Example: `docker ps`

`docker images` - This command is used to list all the available images on your system.

Example: `docker images`

`docker build` - This command is used to build an image from a Dockerfile.

Example: `docker build -t myapp .`

`docker push` - This command is used to push an image to a registry.

Example: `docker push myregistry/myapp`

`docker pull` - This command is used to pull an image from a registry.

Example: `docker pull myregistry/myapp`

`docker stop` - This command is used to stop a running container.

Example: `docker stop mycontainer`

`docker start` - This command is used to start a stopped container.

Example: `docker start mycontainer`

`docker rm` - This command is used to remove a container.

Example: `docker rm mycontainer`

`docker rmi` - This command is used to remove an image.

Example: `docker rmi myimage`

Dockerfile Reference

Docker can build images automatically by reading the instructions from a Dockerfile.

A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image. This page describes the commands you can use in a Dockerfile.

Instruction	Description
-------------	-------------

MAINTAINER: Specify the author of an image.
FROM: Create a new build stage from a base image.
LABEL Add metadata to an image.
RUN: Execute build commands.
EXPOSE: Describe which ports your application is listening on.
ENTRYPOINT: Specify default executable.
CMD: Specify default commands.
COPY: Copy files and directories.
VOLUME: Create volume mounts.
ADD: Add local or remote files and directories.
ARG: Use build-time variables.
ENV: Set environment variables.
HEALTHCHECK: Check a container's health on startup.
ONBUILD: Specify instructions for when the image is used in a build.
SHELL: Set the default shell of an image.
STOPSIGNAL: Specify the system call signal for exiting a container.
USER: Set user and group ID.
WORKDIR: Change working directory.

- **Apache2:**

Apache HTTP Server, often referred to as Apache2, is a free and open-source web server software that allows users to deploy websites and serve web content over the internet.

- **Ubuntu:**

is a free and open-source Linux-based operating system. It's known for its user-friendly interface, making it a popular choice for both beginners and experienced users. No, Ubuntu is not the same as Linux. Linux is the kernel, the core software that manages the computer's hardware, while Ubuntu is a Linux distribution, meaning it's a complete operating system built on top of the Linux kernel.

- **Latest version: Ubuntu 25.04**

Advanced Package Tool (APT)

The "apt-get upgrade" program downloads and installs the most recent packages, replacing any earlier versions that were already on your system. In a nutshell, "apt-get update" updates the package index files, whereas "apt-get upgrade" upgrades the actual packages installed on your system.

By using the -y flag with apt-get install, you can automatically answer yes to any prompts that the installation process might produce, making the process non-interactive. This rule is crucial because a Docker build that requires user interaction can cause problems in automated build systems.

- -f: attempts to fix problems with broken dependencies between program packages.

1. Base Ubuntu Image:

```
docker pull ubuntu:18.04
```

```
docker run -dt --name plainub1 -p 8080:80 Image_ID
```

ifconfig

- Take the ipaddress and check: 192.168.1.145:8080

2. Ubuntu with Apache Image:

```
FROM ubuntu:24.04
```

```
LABEL key=jnareshkumar.pt@gmail.com
```

```
#LABEL maintainer "jnareshkumar.pt@gmail.com"
```

```
RUN apt-get update && apt-get -y install apache2
```

```
EXPOSE 80
```

```
ENTRYPOINT ["/usr/sbin/apachectl"]
```

```
CMD ["-D","FOREGROUND"]
```

```
docker build -t ubapache2 .
```

```
docker run -dt --name ubapache2 -p 8081:80 Image_ID
```

ifconfig

- Take the ipaddress and check: 192.168.1.145:8081

3. Ubuntu with Apache Custom Image:

```
FROM ubuntu:24.04
```

```
LABEL key=jnareshkumar.pt@gmail.com
```

```
#LABEL maintainer "jnareshkumar.pt@gmail.com"
```

```
RUN apt-get update && apt-get -y install apache2
```

```
EXPOSE 80
```

```
ENTRYPOINT ["/usr/sbin/apachectl"]
```

```
CMD ["-D","FOREGROUND"]
```

```
COPY index.html /var/www/html/index.html
```

```
VOLUME /var/www/html
```

```
docker build -t ubapache3 .
```

```
docker run -dt --name ubapache3 -p 8082:80 Image_ID
```

ifconfig

- Take the ipaddress and check: 192.168.1.145:8082

Docker Commands

```
docker ps -a
```

```
docker rm containerID
```

```
docker rmi imagename
```

touch Dockerfile to create a file

nano Dockerfile to write

ctrl+x to save

or echo "Hello World!" > Dockerfile

cat Dockerfile to see the contents

```
docker tag imagename imagename:v1
docker push imagename:v1
docker run -dt --name plainub01 -p 8080:80 152dc042452c
docker logs containername
docker container prune -f : Stopped Containers Only
docker rm -f $(docker ps -aq):      Running + Stopped
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
docker image prune -a -f
docker system prune -a --volumes
```

Kubernetes

1. minikube start
2. minikube status
3. kubectl run demodockuber --image=naresh5021/dockuberdemo:1.0 --port=80
4. docker% kubectl get pods
 kubectl get pods/redis-pod
5. docker% kubectl expose pod demodockuber --name=demodockubersvc --port=80
6. docker% kubectl get svc
7. docker% kubectl port-forward service/demodockubersvc 8181:80

```
kubectl run <pod_name> --image=<your_image_name>
```

```
kubectl get pods
```

```
minikube ssh
```

```
docker ps -a
```

```
docker exec -it <Id_of_container> /bin/bash
```

```
create-redispod.yml:
```

```
apiVersion: v1
```

```
kind:Pod
```

```
metadata:
```

```
  name: redis-pod
```

```
spec:
```

```
  containers:
```

```
    - name: redis-container01
```

```
      image:saravak/redis:latest
```

```
      ports:
```

```
        - containerPort:6379
```

```
kubectl create -f create-redispod.yml
```

```
kubectl run redis-pod -- image=saravak/redis -- port=6379 -- generator=run/v1
```

```
kubectl get pods|egrep -i "^NAME|redis-pod"
```

Docker Project: Build and Run a Simple Web App

Step 1 - Create a new directory mkdir myapp

```
cd myapp
```

Step 2 - Create a file called "index.html" echo "Hello, world!" > index.html

Step 3 - Create a file named Dockerfile touch Dockerfile

Step 4 - Open the "Dockerfile" file in a text editor and add the following lines:

```
FROM nginx
```

```
COPY index.html /usr/share/nginx/html
```

This Dockerfile defines a new Docker image that uses the official nginx image as a base then copy the index.html file to the appropriate location in the image

A Dockerfile is a text file with instructions to build a Docker Image

When we run a Dockerfile, Docker image is created

When we run the docker image, containers are created

Step 5 - Start docker & Build docker image from dockerfile

```
docker build -t myapp .
```

This command builds a new Docker image with the tag "myapp" using the Dockerfile in the current directory.

Step 6 - Run docker container from the image

```
docker run -p 8080:80 myapp
```

This tells Docker to run the myapp container and map port 8080 on your local machine to port 80 inside the container

Step 7 - Access the app

Open a web browser and navigate to <http://localhost:8080> to see the "Hello, world!" message displayed in your web browser.

or curl <http://localhost:8081>

Ansible

Setup Ansible on Windows:

Ansible, primarily designed for Linux, can be set up on Windows using various methods. Below are the most common and effective approaches:

Using Windows Subsystem for Linux (WSL)

This method allows you to run a Linux environment directly on Windows.

Enable WSL: Open PowerShell as Administrator and run:

```
wsl --install
```

Copy

Update Ubuntu: Launch Ubuntu and update the package repository:

```
sudo apt update
```

Copy

Install Ansible: Run the following commands to install Ansible:

```
sudo apt install software-properties-common
```

```
sudo apt-add-repository ppa:ansible/ansible
```

```
sudo apt update
```

```
sudo apt install ansible -y
```

Copy

Verify Installation: Check the installed version with:

```
ansible --version
```

- name: Debug module in ansible

hosts: webservers

tasks:

- name: Print a welcome message

debug: msg="Hello from Ansible Controller"

- name: Execute command on the host server

command: hostname -s

- name: Install packages on the worker node

hosts: webservers

become: true

become_user: root

tasks:

- name: update the apt repo

command: apt-get update

- name: install php on hostserver

package: name=php state=present

- name: Install packages on the worker node

hosts: webservers

become: true

become_user: root

tasks:

- name: update the apt repo

command: apt-get update

- name: install php on hostserver

package: name=php state=present

- name: create a file on the hostserver

file: path=/tmp/myfile state=touch